

SE_46: Backend

I built a website that I have had in mind for a while now and had worked on before. At first it was just a simple travel app where everyone can post the trip they had and review them and say what they liked. I then got the idea of making it more of something where you can track your expenses and add a budget to it and then post it. I built a RESTful API using Node.js, Express, and PostgreSQL that handles user authentication, trip management, expense tracking, and participant management. The backend is deployed on Render and is connected to the deployed frontend and has documentation through Swagger.

The backend implements a complete API with user authentication system with JWT tokens for secure access, trip CRUD operations with date validation and ownership checks, expense management with category support and date range validation, participant management, middleware for authentication and input validation, error handling with error messages, PostgreSQL database with properly structured tables and foreign key constraints, interactive Swagger API for endpoints and deployment configuration for production use on Render.

The main challenges I faced while building the backend of my project were writing tests, date validation logic and the database schema design. Writing test suits with Jest and Supertest was hard since I had to make sure I understood how to properly mock database connections, set up test data, and verify both successful operations and error cases. I needed to test authentication middleware, validate error responses, and ensure endpoints return correct status codes. I was able to do this by making sure each test suit was for one thing (authentication, trips, expenses, middleware), using beforeEach hooks to set up test data consistently and writing tests that cover most edge cases I can think of. Implementing robust date validation for expenses was challenging because expenses need to fall within their trip's date range, and multi-day expenses (like hotels) need proper start and end date validation. This logic appears in both create and update operations. I solved this by writing validation functions that check expense dates against trip dates, ensure end dates come after start dates, and return clear error messages when validation fails. The backend enforces these business rules consistently across all endpoints. The database schema was annoying for me since I had thought I still remembered everything that I had learned a few years ago and I clearly did not since I hadn't made sure of a few key things. I solved this by relooking through what I had learned before and made sure I implemented proper foreign key constraints with CASCADE DELETE to maintain referential integrity, creating junction tables for many-to-many relationships, and using indices on frequently queried columns to optimize performance.

I am very happy with my work in the backend and I also think I could have done a bit more but I am very happy with the point I have reached now with the backend, I learned a lot of things that will definitely help me in the future. I think a fair assessment for me would be around the high end of level 1 or beginning of level 2.